The Unified Modeling Language

Appendix C

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Unified Modeling Language

- The Unified Modeling Language, or UML, is a modeling language used to express object-oriented designs
- A use case is a set of textual scenarios or stories that describe the proposed solution, but does not describe how the system works
- A use case is not object-oriented, but it is an important part of the analysis of the problem.
- The main success scenario (or “happy path”) describes how the system satisfies the goals of the user when everything goes smoothly.
- A scenario is written from the perspective of the user of the system.
- Alternate scenarios should be written only for exception cases.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Sequence Diagram

- A UML sequence diagram, or interaction diagram,
  - shows how two or more objects will interact with one another over time within a single scenario (process flow), after use cases and potential objects are created.
  - allows you to visualize the messages sent among the objects in a scenario and the order in which those messages happen
  - defines the responsibilities of the objects:
    - What must the object "remember," or keep track of, and
    - what must the object do for other objects?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Sequence Diagram

Show objects left to right

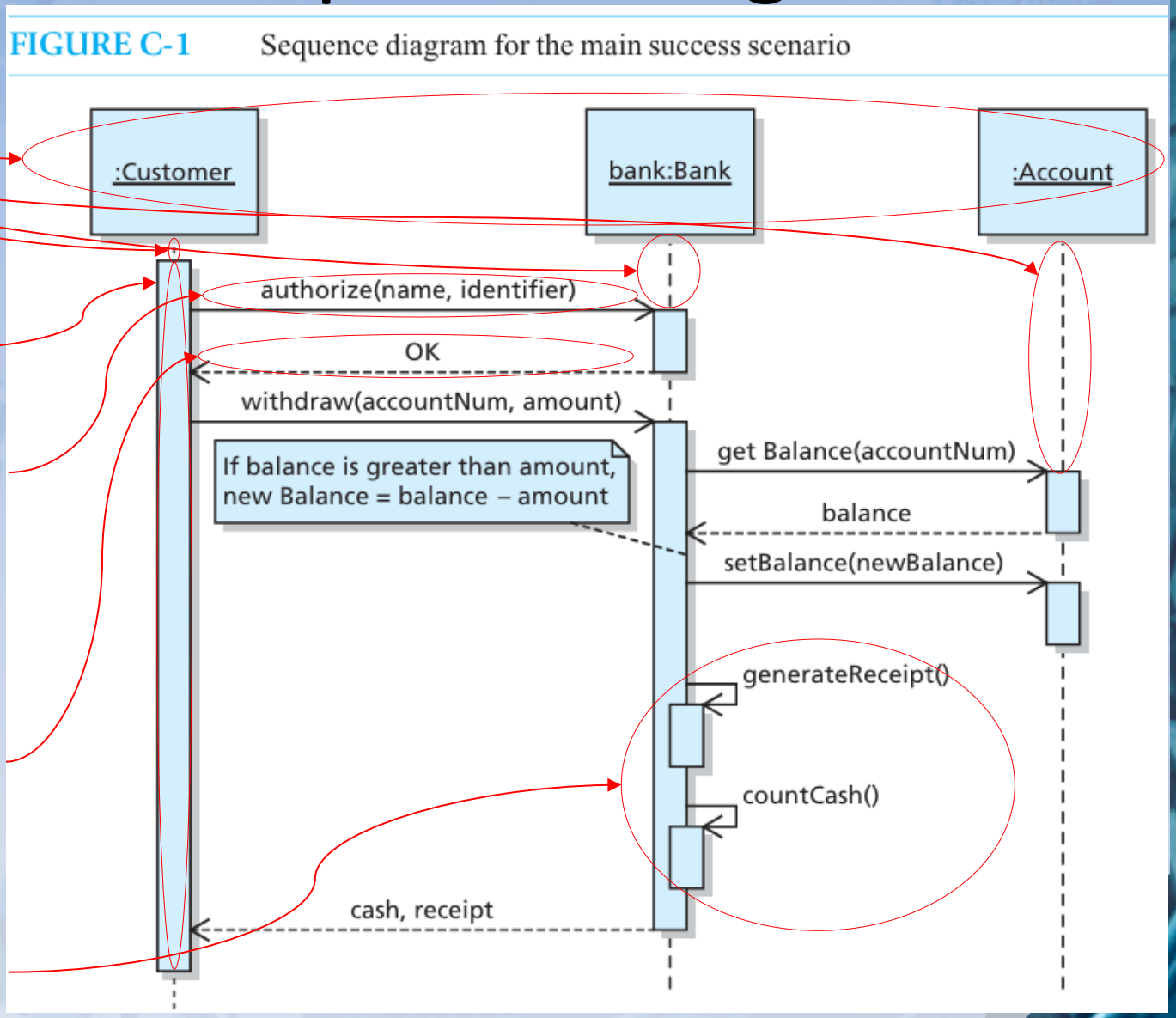
Each object has a lifeline (vertical dashed line)

Each lifeline can have 1 or more activation bars (open box on lifeline)

To send a message between objects, draw a solid arrow from the activation bar of the calling object to the top of a new activation bar on the lifeline of the called object, with message and parameters

Task completes and returns (dashed arrow w/ variable)

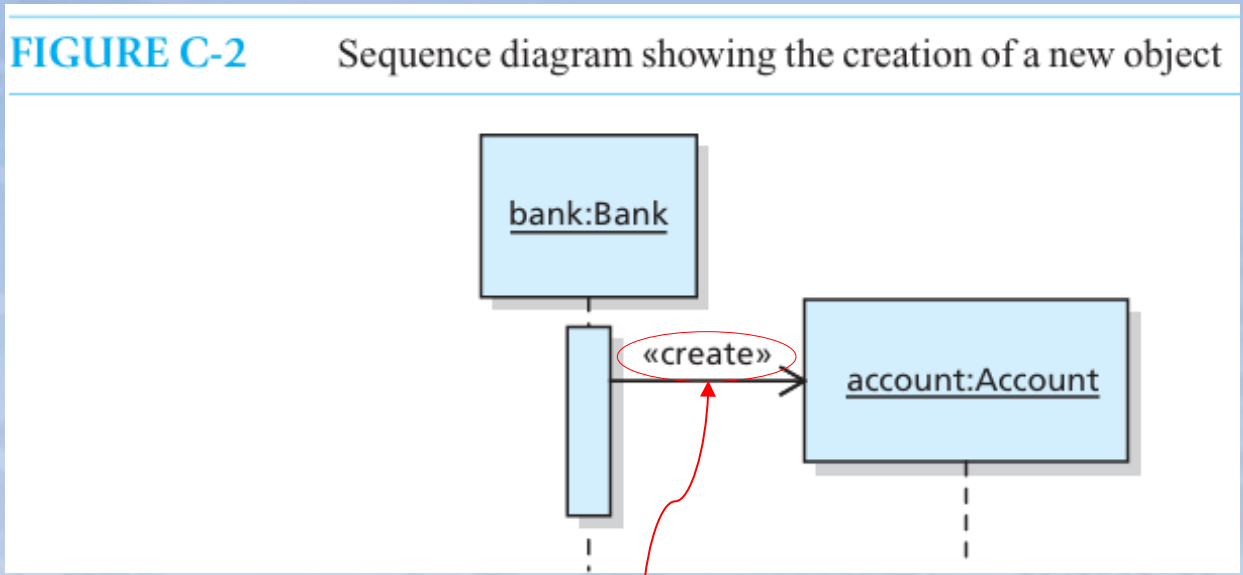
Objects may send messages to itself



© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Unified Modeling Language

In Figure C-2, a new account is created.



Known as stereotype, which use left and right guillemets (<< and >>)

A stereotype is used to identify a unique characteristic of an element in any UML diagram.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Sequence Diagram

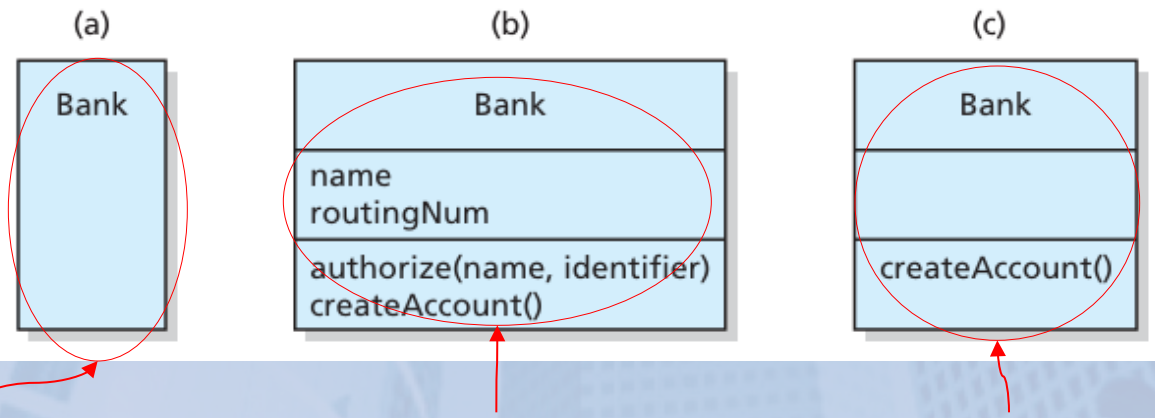
- Is a design document that gives guidance on several levels to the programmer implementing a solution
- Shows which objects collaborate to solve a particular use-case scenario
- Shows the synchronous nature of the messages that are passed between the objects in the scenario, providing the order in which tasks are accomplished
- Shows which object is responsible for what task or tasks by handling a particular message
- Provides a visual representation of the flow of messages sent between objects during a single scenario

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Class Diagram

The class diagram (static diagram) shows the attributes and operations of an individual class and how multiple classes are related to one another in a language-independent way.

FIGURE C-3 Three possible class diagrams for a class of banks



Class with a name, with no attributes or operations

Class with attributes in the middle, and operations at the bottom

Class with no attributes (empty middle section), and operations at the bottom

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Class Diagram

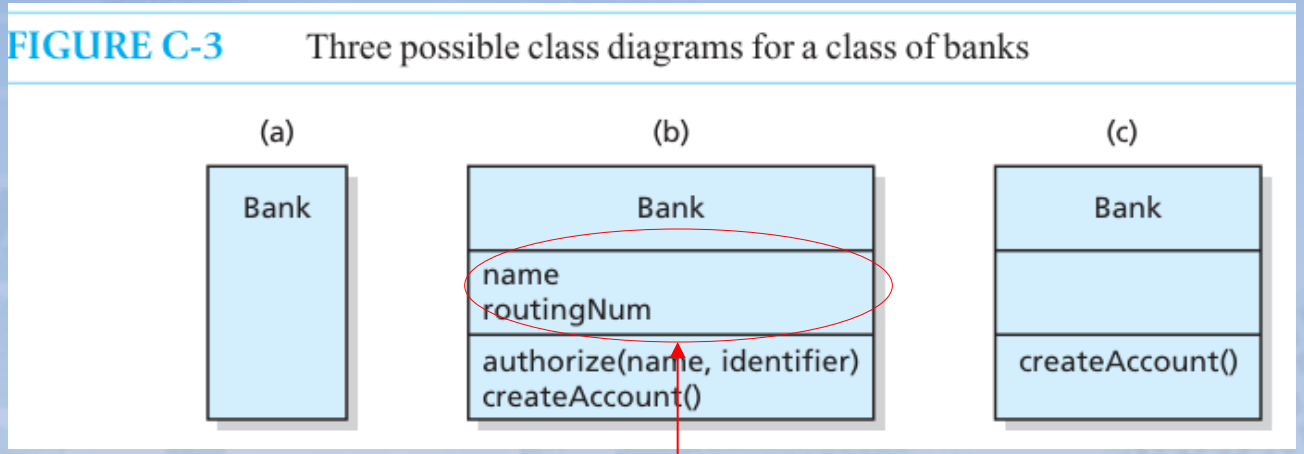
- The UML syntax for the attributes of a class is

  [visibility] name [:type][=defaultValue] [{property}]

- where
  - All elements within square brackets are optional.
  - visibility can be
    - + for public accessibility, (open for all)
    - - for private accessibility (default, if omitted; open to member functions and friends), or
    - # for protected accessibility (open to member functions and friends, and member functions/friends of derived classes) .
  - name is the name of the attribute. (Required.)
  - type is the data type of the attribute. (e.g., integer, string)
  - defaultValue is the default value of the attribute.
  - property (changeable or frozen/constant).

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Class Diagram



- The attributes for the Bank class in Figure C-3b could now be written as
  - –name: string
  - –routingNum: integer

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Class Diagram

- The UML syntax for a class's operations is:

  [visibility] name ([parameterList]) [:type] [{property}]

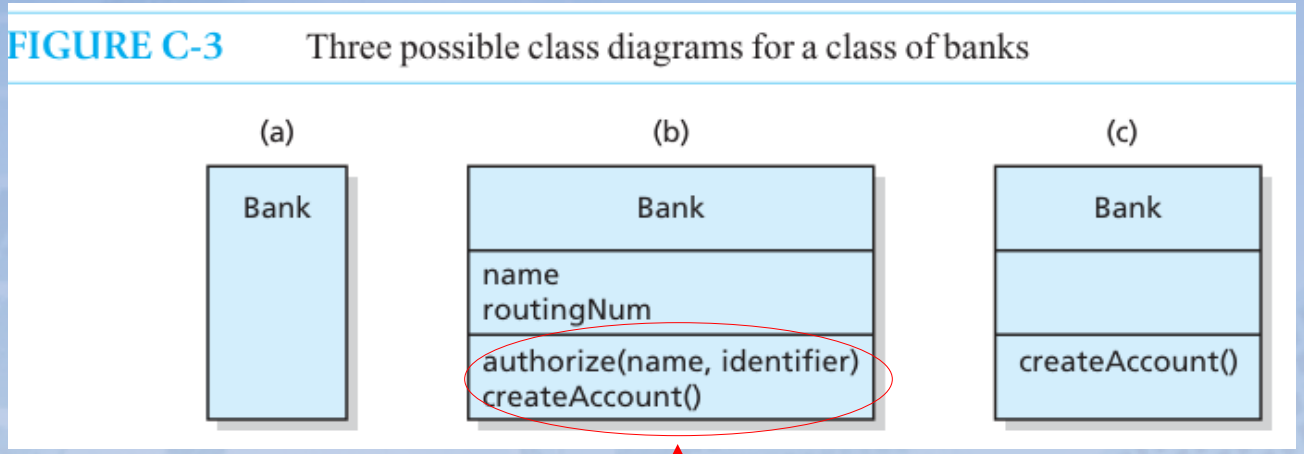
- where
  - All elements within square brackets are optional.
  - Visibility: +public, -private or #protected, (default, if omitted, to public).
  - name is the name of the operation (Required)
  - type is the data type of the result returned from the operation (e.g., void, integer, float,…)
  - property - The only property relevant to us is query, which indicates the operation will not modify any of the data.
  - parameterList is either empty—in which case the parentheses are still required—or contains a comma-delimited list of parameters to the operation. The syntax for each parameter looks like the following:

    [direction] name:type [=defaultValue]

- where
  - All elements within square brackets are optional.
  - direction shows whether the parameter is used for input (in), output (out), or both input and output (inout).
  - name is the name of the parameter.
  - type is the data type of the parameter. (e.g., integer, float,…)
  - defaultValue is the value of this parameter if no corresponding actual argument is provided when this operation is called.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The UML Class Diagram



- The Bank class operations in Figure C-3b could be written as

  +authorize(in name: string, in identifier: string): void {query}

  -createAccount()
- What accessibility does authorize have?
  - public
- What accessibility does createAccount have?
  - private

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

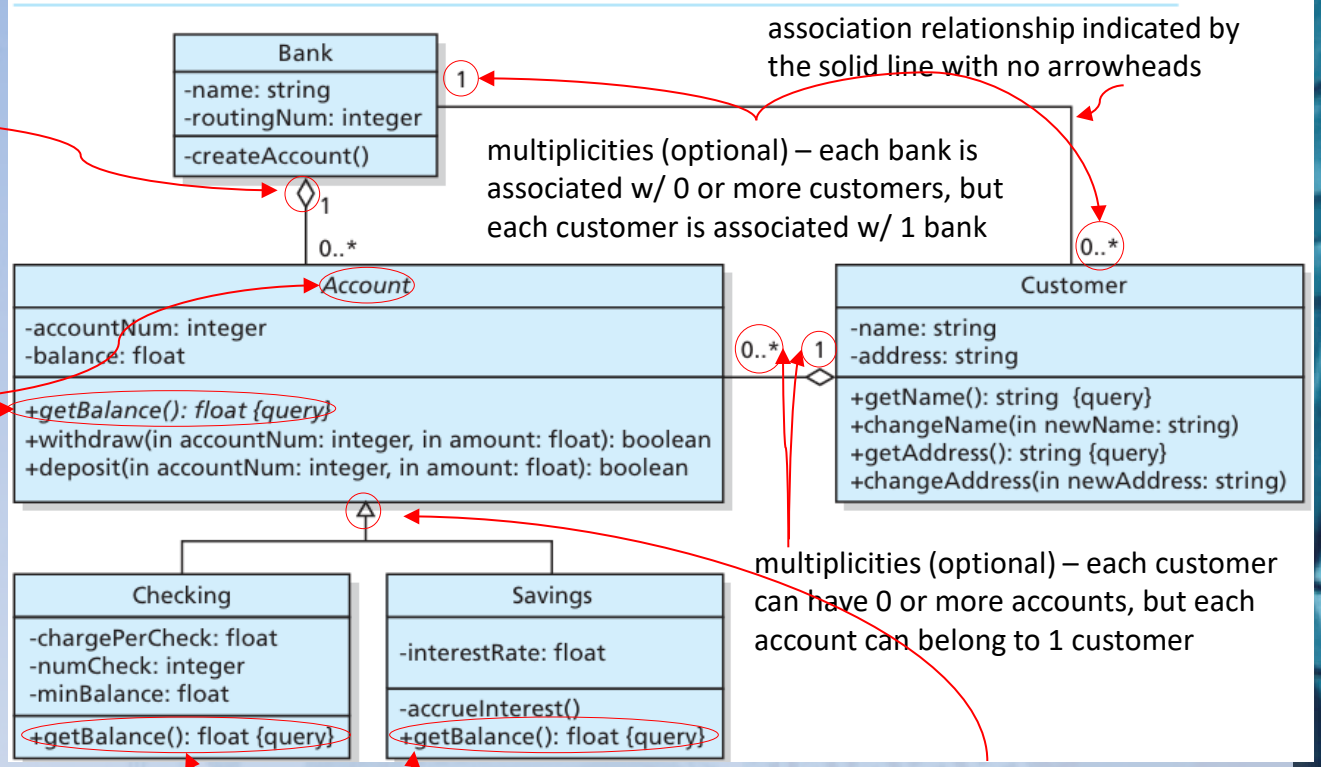
The UML Class Diagram

5 classes: 1) Bank, 2) Customer, 3) Account, 4) Checking and 5) Savings

- Bank and Account classes have **aggregation** (**containment**) relationship
- The open diamond head denotes containing class; banks live longer than accounts
- Filled in diamond head is a **composition** relationship, where lifetimes are the same

- Account class name is in italic, which denotes an abstract base class
- The getBalance operation in the Account class is also in italic. This indicates that getBalance is not implemented in the Account class but must be implemented within the descendant class.

FIGURE C-4 A UML class diagram of a banking system



- Checking and Savings classes each implement the inherited getBalance operation.
- This operation will take on polymorphic behavior when called from a parent class object.

- The Account class is related to the Checking and Savings classes via **generalization** (inheritance) relationship, indicated by the open triangular arrowhead
- Checking and Savings inherit the attributes and operations of the Account class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

The Unified Modeling Language

- You should use the UML to quickly draw various aspects of the current problem to better understand it during OOA and to create a set of objects during OOD that can collaborate to solve this problem.
- Often small teams of programmers will draw several UML diagrams on a whiteboard and, after working out the kinks in the diagrams, take digital pictures of them for future reference.
- These design diagrams are considered an artifact, or document, associated with the analysis and design. However, the ultimate implementation—that is, the solution—might not exactly match the original design. Thus, the design diagrams might not document the solution.
- UML can conceptually model a problem domain in terms of software objects, independently of a programming language.
- The UML enables the members of a programming team to communicate visually with one another, thereby gaining a common understanding of the system that is being built.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Appendix C

© 2017 Pearson Education, Hoboken, NJ. All rights reserved